

Step 1: Problem Understanding

Before touching the data, understand the business problem.

Problem Statement

A real-estate company wants to predict the selling price of houses based on various house features.

Questions to Answer

1. Which features affect house prices?
2. Does house size affect price?
3. Does neighborhood affect price?
4. Which features are most important for prediction?

Step 2: Data Collection / Loading

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
```

```
df = pd.read_csv("train.csv")
```

```
df.head()
```

	Id	MSSubClass	MSZoning	LotFrontage	LotArea	Street	Alley	LotShape	LandContour	Utilities	...	PoolArea	PoolQC	Fence
0	1	60	RL	65.0	8450	Pave	NaN	Reg	Lvl	AllPub	...	0	NaN	NaN
1	2	20	RL	80.0	9600	Pave	NaN	Reg	Lvl	AllPub	...	0	NaN	NaN
2	3	60	RL	68.0	11250	Pave	NaN	IR1	Lvl	AllPub	...	0	NaN	NaN
3	4	70	RL	60.0	9550	Pave	NaN	IR1	Lvl	AllPub	...	0	NaN	NaN
4	5	60	RL	84.0	14260	Pave	NaN	IR1	Lvl	AllPub	...	0	NaN	NaN

5 rows × 81 columns

Why?**To verify:**

- Dataset loaded successfully.
- Columns are visible.
- Data format looks correct.

Step 3: Basic Data Inspection

```
df.shape
```

```
(1460, 81)
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1460 entries, 0 to 1459
Data columns (total 81 columns):
#   Column          Non-Null Count  Dtype
---  -
0   Id               1460 non-null  int64
1   MSSubClass       1460 non-null  int64
2   MSZoning         1460 non-null  object
3   LotFrontage     1201 non-null  float64
4   LotArea         1460 non-null  int64
5   Street          1460 non-null  object
6   Alley           91 non-null    object
7   LotShape        1460 non-null  object
8   LandContour     1460 non-null  object
```

```
9 Utilities 1460 non-null object
10 LotConfig 1460 non-null object
11 LandSlope 1460 non-null object
12 Neighborhood 1460 non-null object
13 Condition1 1460 non-null object
14 Condition2 1460 non-null object
15 BldgType 1460 non-null object
16 HouseStyle 1460 non-null object
17 OverallQual 1460 non-null int64
18 OverallCond 1460 non-null int64
19 YearBuilt 1460 non-null int64
20 YearRemodAdd 1460 non-null int64
21 RoofStyle 1460 non-null object
22 RoofMatl 1460 non-null object
23 Exterior1st 1460 non-null object
24 Exterior2nd 1460 non-null object
25 MasVnrType 588 non-null object
26 MasVnrArea 1452 non-null float64
27 ExterQual 1460 non-null object
28 ExterCond 1460 non-null object
29 Foundation 1460 non-null object
30 BsmtQual 1423 non-null object
31 BsmtCond 1423 non-null object
32 BsmtExposure 1422 non-null object
33 BsmtFinType1 1423 non-null object
34 BsmtFinSF1 1460 non-null int64
35 BsmtFinType2 1422 non-null object
36 BsmtFinSF2 1460 non-null int64
37 BsmtUnfSF 1460 non-null int64
38 TotalBsmtSF 1460 non-null int64
39 Heating 1460 non-null object
40 HeatingQC 1460 non-null object
41 CentralAir 1460 non-null object
42 Electrical 1459 non-null object
43 1stFlrSF 1460 non-null int64
44 2ndFlrSF 1460 non-null int64
45 LowQualFinSF 1460 non-null int64
46 GrLivArea 1460 non-null int64
47 BsmtFullBath 1460 non-null int64
48 BsmtHalfBath 1460 non-null int64
49 FullBath 1460 non-null int64
50 HalfBath 1460 non-null int64
51 BedroomAbvGr 1460 non-null int64
```

df.describe()

	Id	MSSubClass	LotFrontage	LotArea	OverallQual	OverallCond	YearBuilt	YearRemodAdd	MasVnrArea	Bsm
count	1460.000000	1460.000000	1201.000000	1460.000000	1460.000000	1460.000000	1460.000000	1460.000000	1452.000000	1460
mean	730.500000	56.897260	70.049958	10516.828082	6.099315	5.575342	1971.267808	1984.865753	103.685262	443
std	421.610009	42.300571	24.284752	9981.264932	1.382997	1.112799	30.202904	20.645407	181.066207	456
min	1.000000	20.000000	21.000000	1300.000000	1.000000	1.000000	1872.000000	1950.000000	0.000000	0
25%	365.750000	20.000000	59.000000	7553.500000	5.000000	5.000000	1954.000000	1967.000000	0.000000	0
50%	730.500000	50.000000	69.000000	9478.500000	6.000000	5.000000	1973.000000	1994.000000	0.000000	383
75%	1095.250000	70.000000	80.000000	11601.500000	7.000000	6.000000	2000.000000	2004.000000	166.000000	712
max	1460.000000	190.000000	313.000000	215245.000000	10.000000	9.000000	2010.000000	2010.000000	1600.000000	5644

8 rows × 38 columns

df.sample(5)

	Id	MSSubClass	MSZoning	LotFrontage	LotArea	Street	Alley	LotShape	LandContour	Utilities	...	PoolArea	PoolQC	Fe
340	341	60	RL	85.0	14191	Pave	NaN	Reg	Lvl	AllPub	...	0	NaN	
25	26	20	RL	110.0	14230	Pave	NaN	Reg	Lvl	AllPub	...	0	NaN	
1128	1129	60	RL	59.0	11796	Pave	NaN	IR1	Lvl	AllPub	...	0	NaN	
677	678	30	RL	52.0	9022	Pave	NaN	Reg	Lvl	AllPub	...	0	NaN	
104	105	50	RM	NaN	7758	Pave	NaN	Reg	Lvl	AllPub	...	0	NaN	

5 rows × 81 columns

Step 4: Understanding Features & Data Types

```
num_cols = df.select_dtypes(
    exclude='object'
).columns

cat_cols = df.select_dtypes(
    include='object'
).columns
```

```
df['MSZoning']
```

MSZoning	
0	RL
1	RL
2	RL
3	RL
4	RL
...	...
1455	RL
1456	RL
1457	RL
1458	RL
1459	RL

1460 rows × 1 columns

dtype: object

```
for col in cat_cols:
    print(col, df[col].unique())
```

```
MSZoning ['RL' 'RM' 'C (all)' 'FV' 'RH']
Street ['Pave' 'Grv1']
Alley [nan 'Grv1' 'Pave']
LotShape ['Reg' 'IR1' 'IR2' 'IR3']
LandContour ['Lvl' 'Bnk' 'Low' 'HLS']
Utilities ['AllPub' 'NoSeWa']
LotConfig ['Inside' 'FR2' 'Corner' 'CulDSac' 'FR3']
LandSlope ['Gtl' 'Mod' 'Sev']
Neighborhood ['CollgCr' 'Veenker' 'Crawfor' 'NoRidge' 'Mitchel' 'Somerst' 'NWAmes'
'OldTown' 'BrkSide' 'Sawyer' 'NridgHt' 'NAmes' 'SawyerW' 'IDOTRR'
'MeadowV' 'Edwards' 'Timber' 'Gilbert' 'StoneBr' 'ClearCr' 'NPKVill'
'Blmngtn' 'BrDale' 'SWISU' 'Blueste']
Condition1 ['Norm' 'Feedr' 'PosN' 'Artery' 'RAAe' 'RRNn' 'RRAn' 'PosA' 'RRNe']
Condition2 ['Norm' 'Artery' 'RRNn' 'Feedr' 'PosN' 'PosA' 'RRAn' 'RAAe']
BldgType ['1Fam' '2fmCon' 'Duplex' 'TwnhsE' 'Twnhs']
HouseStyle ['2Story' '1Story' '1.5Fin' '1.5Unf' 'SFoyer' 'SLvl' '2.5Unf' '2.5Fin']
RoofStyle ['Gable' 'Hip' 'Gambrel' 'Mansard' 'Flat' 'Shed']
RoofMatl ['CompShg' 'WdShngl' 'Metal' 'WdShake' 'Membran' 'Tar&Grv' 'Roll'
'ClyTile']
Exterior1st ['VinylSd' 'MetalSd' 'Wd Sdng' 'HdBoard' 'BrkFace' 'WdShing' 'CemntBd'
'Plywood' 'AsbShng' 'Stucco' 'BrkComm' 'AsphShn' 'Stone' 'ImStucc'
'CBlock']
Exterior2nd ['VinylSd' 'MetalSd' 'Wd Shng' 'HdBoard' 'Plywood' 'Wd Sdng' 'CmentBd'
'BrkFace' 'Stucco' 'AsbShng' 'Brk Cmn' 'ImStucc' 'AsphShn' 'Stone'
'Other' 'CBlock']
MasVnrType ['BrkFace' nan 'Stone' 'BrkCmn']
ExterQual ['Gd' 'TA' 'Ex' 'Fa']
ExterCond ['TA' 'Gd' 'Fa' 'Po' 'Ex']
Foundation ['PConc' 'CBlock' 'BrkTil' 'Wood' 'Slab' 'Stone']
BsmtQual ['Gd' 'TA' 'Ex' nan 'Fa']
BsmtCond ['TA' 'Gd' nan 'Fa' 'Po']
BsmtExposure ['No' 'Gd' 'Mn' 'Av' nan]
BsmtFinType1 ['GLQ' 'ALQ' 'Unf' 'Rec' 'BLQ' nan 'LwQ']
BsmtFinType2 ['Unf' 'BLQ' nan 'ALQ' 'Rec' 'LwQ' 'GLQ']
Heating ['GasA' 'GasW' 'Grav' 'Wall' 'OthW' 'Floor']
HeatingQC ['Ex' 'Gd' 'TA' 'Fa' 'Po']
CentralAir ['Y' 'N']
Electrical ['SBrkr' 'FuseF' 'FuseA' 'FuseP' 'Mix' nan]
```

```

KitchenQual ['Gd' 'TA' 'Ex' 'Fa']
Functional ['Typ' 'Min1' 'Maj1' 'Min2' 'Mod' 'Maj2' 'Sev']
FireplaceQu [nan 'TA' 'Gd' 'Fa' 'Ex' 'Po']
GarageType ['Attchd' 'Detchd' 'BuiltIn' 'CarPort' nan 'Basment' '2Types']
GarageFinish ['RFn' 'Unf' 'Fin' nan]
GarageQual ['TA' 'Fa' 'Gd' nan 'Ex' 'Po']
GarageCond ['TA' 'Fa' nan 'Gd' 'Po' 'Ex']
PavedDrive ['Y' 'N' 'P']
PoolQC [nan 'Ex' 'Fa' 'Gd']
Fence [nan 'MnPrv' 'GdWo' 'GdPrv' 'MnWw']
MiscFeature [nan 'Shed' 'Gar2' 'Othr' 'TenC']
SaleType ['WD' 'New' 'COD' 'ConLD' 'ConLI' 'CWD' 'ConLw' 'Con' 'Oth']
SaleCondition ['Normal' 'Abnorml' 'Partial' 'AdjLand' 'Alloca' 'Family']

```

Step 5: Handling Missing values

```
df.isnull().sum()
```

```
df.isnull().sum().sort_values(ascending=False)
```

```

0
PoolQC      1453
MiscFeature  1406
Alley       1369
Fence       1179
MasVnrType   872
...         ...
MoSold       0
YrSold       0
SaleType     0
SaleCondition 0
SalePrice    0

```

81 rows × 1 columns

dtype: int64

```

missing = df.isnull().sum().sort_values(
    ascending=False
)

missing[missing > 0]

```

```

0
FireplaceQu    690
LotFrontage    259
GarageYrBlt     81
GarageQual      81
GarageFinish    81
GarageType      81
GarageCond      81
BsmtFinType2    38
BsmtExposure    38
BsmtQual        37
BsmtCond        37
BsmtFinType1    37
MasVnrArea       8
Electrical       1

```

dtype: int64

```

# IF the particular is having more than 50% of the data as null values
# then drop that column

```

```
missing_percent[missing_percent >= 50]
```

```

0
Alley    93.767123
MasVnrType  59.726027
PoolQC   99.520548
Fence    80.753425
MiscFeature  96.301370

```

dtype: float64

```

missing_percent = (
    df.isnull().sum() /
    len(df)
) * 100

missing_percent.sort_values(
    ascending=False
)

missing_percent[missing_percent >= 50]

```

```

0
Alley    93.767123
MasVnrType  59.726027
PoolQC   99.520548
Fence    80.753425
MiscFeature  96.301370

```

dtype: float64

```
df.drop(['Alley', 'MasVnrType', 'PoolQC', 'Fence', 'MiscFeature'], axis=1, inplace=True)
```

```

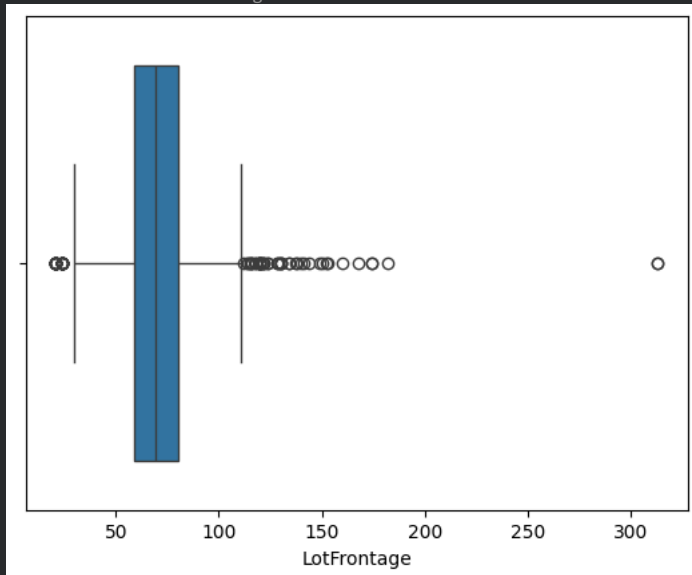
sns.boxplot(
    x=df['LotFrontage']

```

```
)

# Since the LotFrontage is having the many outliers then we can't impute that with mean
```

```
<Axes: xlabel='LotFrontage'>
```



```
df['LotFrontage'].fillna(
    df['LotFrontage'].median(),
    inplace=True
)
```

/tmp/ipykernel_5495/974364480.py:1: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chain. The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting is a copy.
For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value, inplace=True)

```
df['LotFrontage'].fillna(
```

```
df
```

```
df['GarageType'].fillna(
    df['GarageType'].mode()[0],
    inplace=True
)
```

/tmp/ipykernel_5495/2731032266.py:1: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chain. The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting is a copy.
For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value, inplace=True)

```
df['GarageType'].fillna(
```

Step 6: Handling Duplicate Data

```
df.duplicated().sum()
```

```
np.int64(0)
```

```
df.drop_duplicates(
    inplace=True
)
```

Step 7: Data Cleaning

```
df
```

	MSSubClass	MSZoning	LotFrontage	LotArea	Street	LotShape	LandContour	Utilities	LotConfig	LandSlope	...	EnclosedPo
0	60	RL	65.0	8450	Pave	Reg	Lvl	AllPub	Inside	Gtl	...	
1	20	RL	80.0	9600	Pave	Reg	Lvl	AllPub	FR2	Gtl	...	
2	60	RL	68.0	11250	Pave	IR1	Lvl	AllPub	Inside	Gtl	...	
3	70	RL	60.0	9550	Pave	IR1	Lvl	AllPub	Corner	Gtl	...	
4	60	RL	84.0	14260	Pave	IR1	Lvl	AllPub	FR2	Gtl	...	
...	...	...	...	...	...	...	...	...	...	...	...	
1455	60	RL	62.0	7917	Pave	Reg	Lvl	AllPub	Inside	Gtl	...	
1456	20	RL	85.0	13175	Pave	Reg	Lvl	AllPub	Inside	Gtl	...	
1457	70	RL	66.0	9042	Pave	Reg	Lvl	AllPub	Inside	Gtl	...	
1458	20	RL	68.0	9717	Pave	Reg	Lvl	AllPub	Inside	Gtl	...	
1459	20	RL	75.0	9937	Pave	Reg	Lvl	AllPub	Inside	Gtl	...	

1460 rows × 75 columns

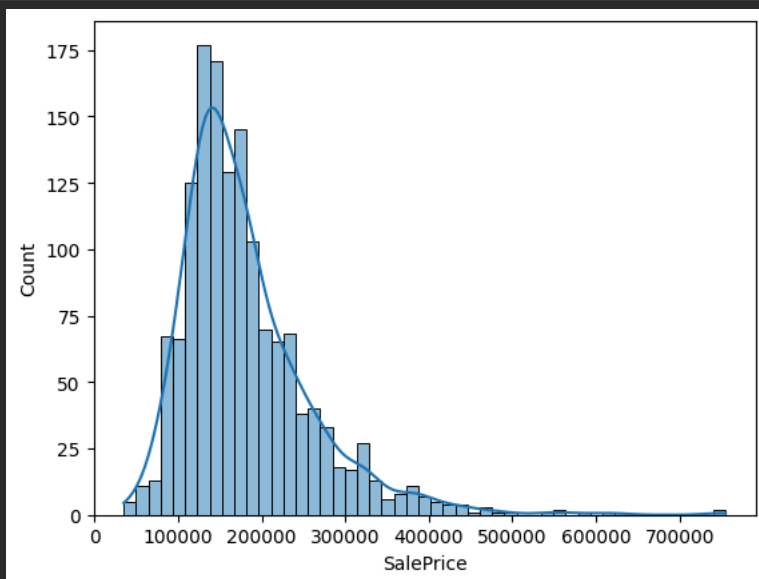
```
df.drop(
    columns=['Id'],
    inplace=True
)
```

Step 8: Univariate Analysis

```
# For the numerical columns
# Shape
# Skewness
# Peaks
sns.histplot(
    df['SalePrice'],
    kde=True
)

plt.show()

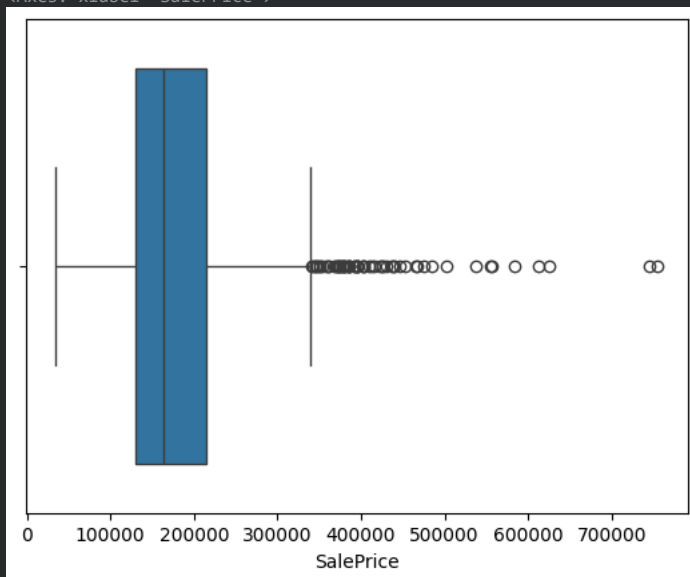
# Insights
# The saleprice column is right skewed data which have some outliers at the right side..
# Most of the price is distributed from 100000 to 300000
# The peaks are between 100000 to 200000
```



```
sns.boxplot(
    x=df['SalePrice']
)
```

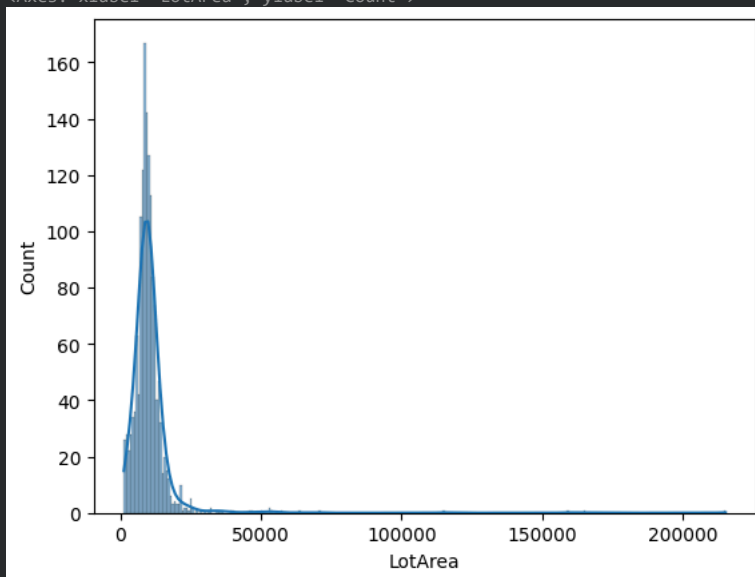
```
# SalePrice has some outliers at the right side which mean the people who are willing
# to sell their houses they are elites or exceptional house..whose price is very high
# as compare to the other data
```

```
<Axes: xlabel='SalePrice'>
```



```
sns.histplot(
    df['LotArea'],
    kde=True
)
```

```
<Axes: xlabel='LotArea', ylabel='Count'>
```

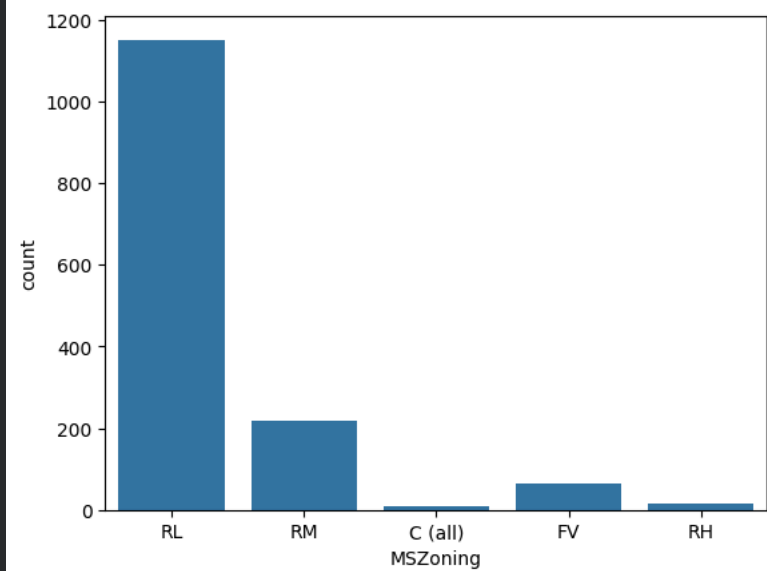


Categorical Columns

```
sns.countplot(
    x=df['MSZoning']
)
```

```
# In this column the category RL has the highest count than others
# C(all) category has very less count minimum count
```

<Axes: xlabel='MSZoning', ylabel='count'>



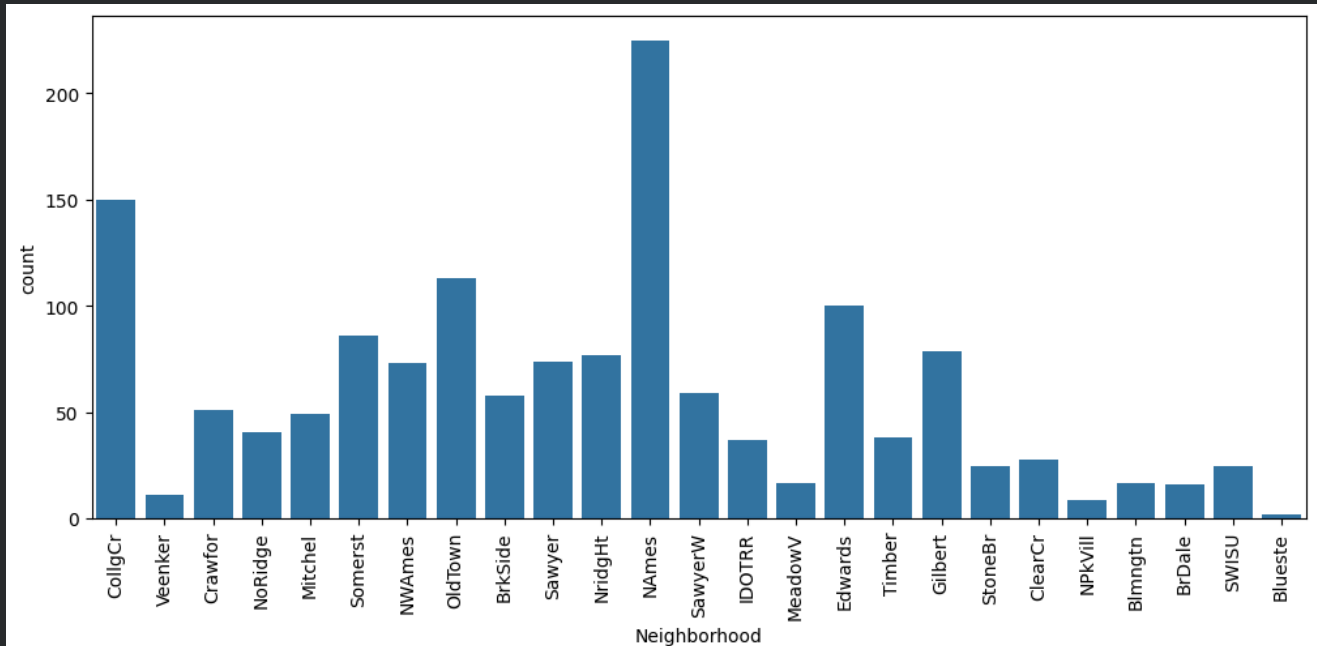
```
plt.figure(figsize=(12,5))

sns.countplot(
    x=df['Neighborhood']
)

plt.xticks(rotation=90)

plt.show()

# Names Category has the highest count among others
# Lowest count has Blueste
```



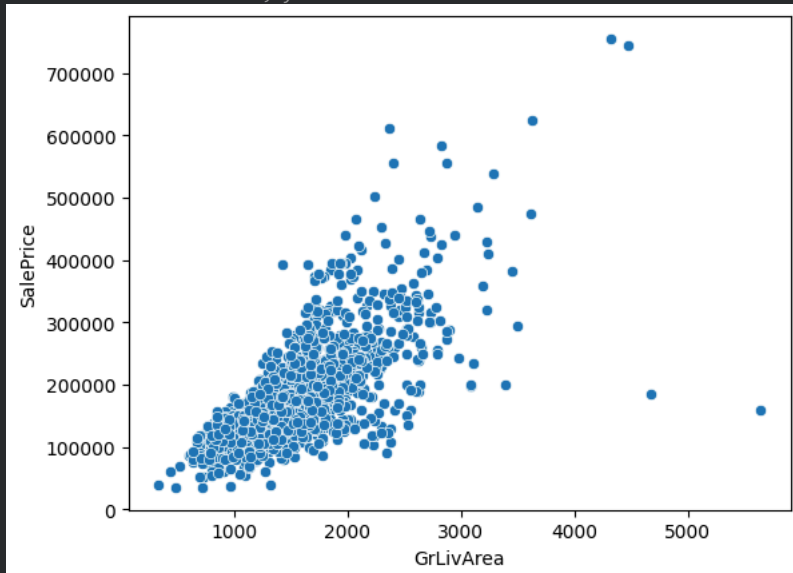
Step 9: Bivariate Analysis

A. Numerical vs Numerical

```
sns.scatterplot(
    x='GrLivArea',
    y='SalePrice',
    data=df
)
```

```
# This feature is important because as the LivArea is increasing then the price is also increasing
# So, It means we to keep this feature
```

```
<Axes: xlabel='GrLivArea', ylabel='SalePrice'>
```



```
df.corr(
    numeric_only=True
)['SalePrice'].sort_values(
    ascending=False
)
```

From here we can select the important features from the data to train a model which will affect the model training..The data or the column is nearer to the 1 they're the important features(we have to keep that feature)

Direct relation

The columns which are around 0 they're the least important they doesn't show any of the relation..

The columns which are in negative then they're the also the important features to the find the pattern..

It means that they're indirectly correlated..

	SalePrice
SalePrice	1.000000
OverallQual	0.790982
GrLivArea	0.708624
GarageCars	0.640409
GarageArea	0.623431
TotalBsmtSF	0.613581
1stFlrSF	0.605852
FullBath	0.560664
TotRmsAbvGrd	0.533723
YearBuilt	0.522897
YearRemodAdd	0.507101
GarageYrBlt	0.486362
MasVnrArea	0.477493
Fireplaces	0.466929
BsmtFinSF1	0.386420
LotFrontage	0.334771
WoodDeckSF	0.324413
2ndFlrSF	0.319334
OpenPorchSF	0.315856
HalfBath	0.284108
LotArea	0.263843
BsmtFullBath	0.227122
BsmtUnfSF	0.214479
BedroomAbvGr	0.168213
ScreenPorch	0.111447
PoolArea	0.092404
MoSold	0.046432
3SsnPorch	0.044584
BsmtFinSF2	-0.011378
BsmtHalfBath	-0.016844
MiscVal	-0.021190
LowQualFinSF	-0.025606
YrSold	-0.028923
OverallCond	-0.077856
MSSubClass	-0.084284
EnclosedPorch	-0.128578
KitchenAbvGr	-0.135907

dtype: float64

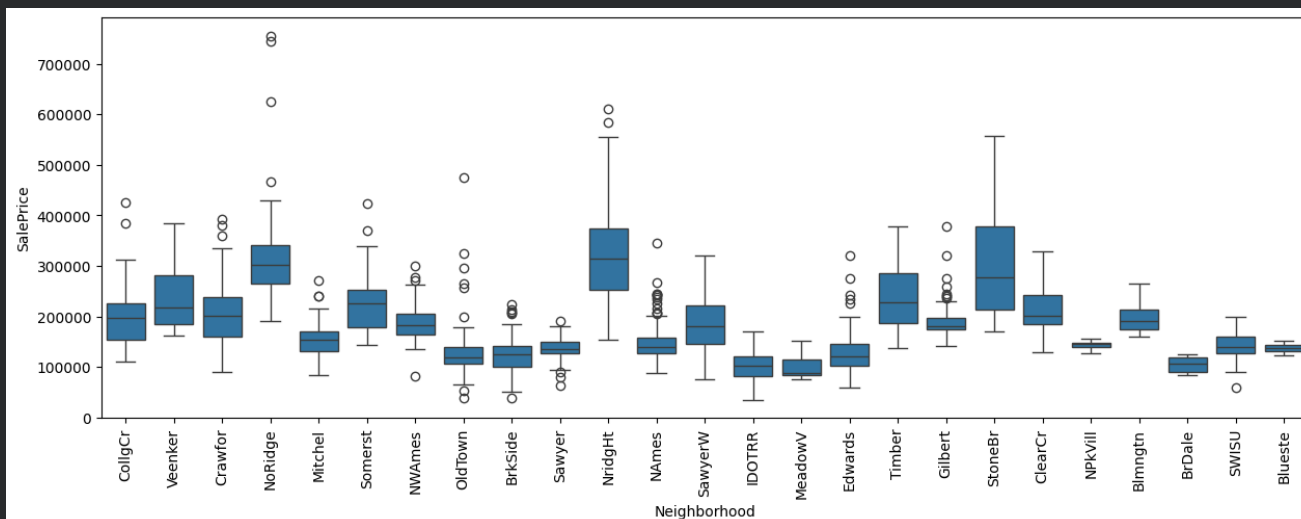
B. Numerical vs Categorical

```
plt.figure(figsize=(15,5))

sns.boxplot(
    x='Neighborhood',
    y='SalePrice',
    data=df
)
```

```
plt.xticks(rotation=90)
plt.show()
```

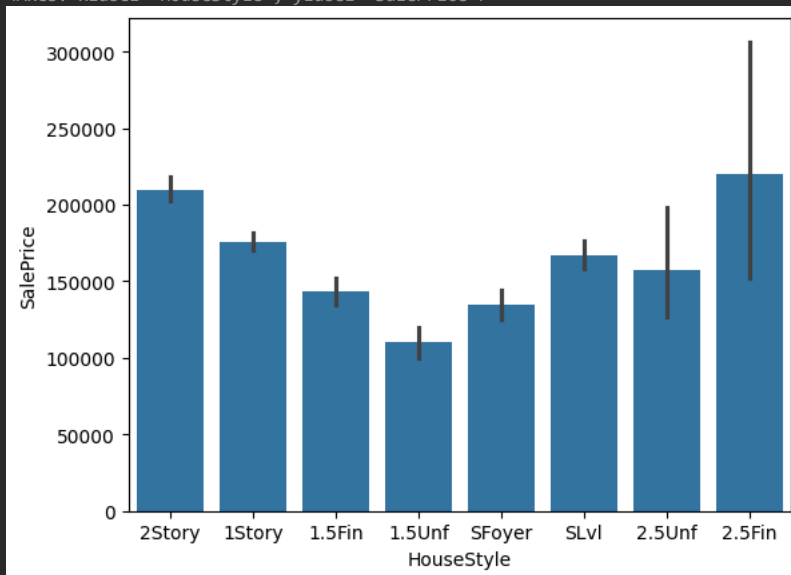
```
# There is Noiridge Category into the neighbourhood who sold their house
# at more than 7 lakh and they're most expensive than others
```



```
sns.barplot(
    x='HouseStyle',
    y='SalePrice',
    data=df
)
```

```
# This shows the frequency as well the data consistency that if the black line shorter then the
# the must be more predictable..
# But if there is longer lines then that category must be inconsistent so, that it can't be predicted..
```

```
<Axes: xlabel='HouseStyle', ylabel='SalePrice'>
```

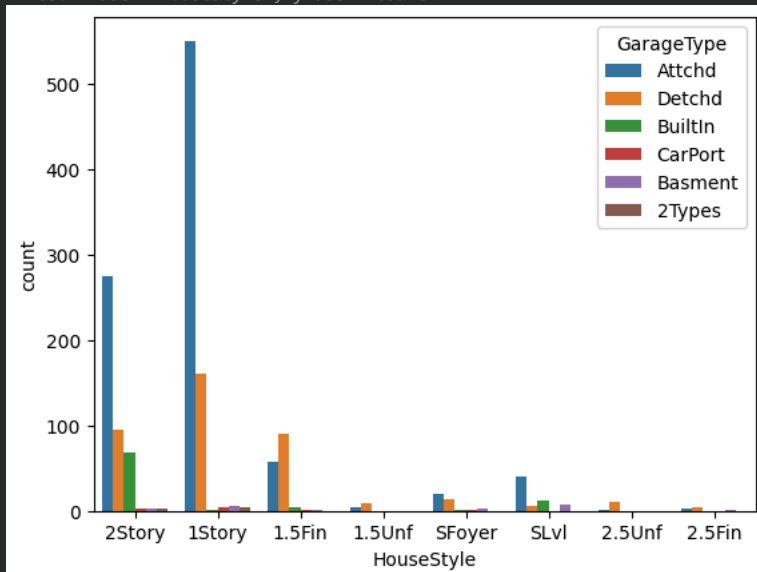


C. Categorical vs Categorical

```
sns.countplot(
    x='HouseStyle',
    hue='GarageType',
    data=df
)
```

```
# from this graph we found that attchd garatype is more than the others in the 1story house style..
```

```
<Axes: xlabel='HouseStyle', ylabel='count'>
```



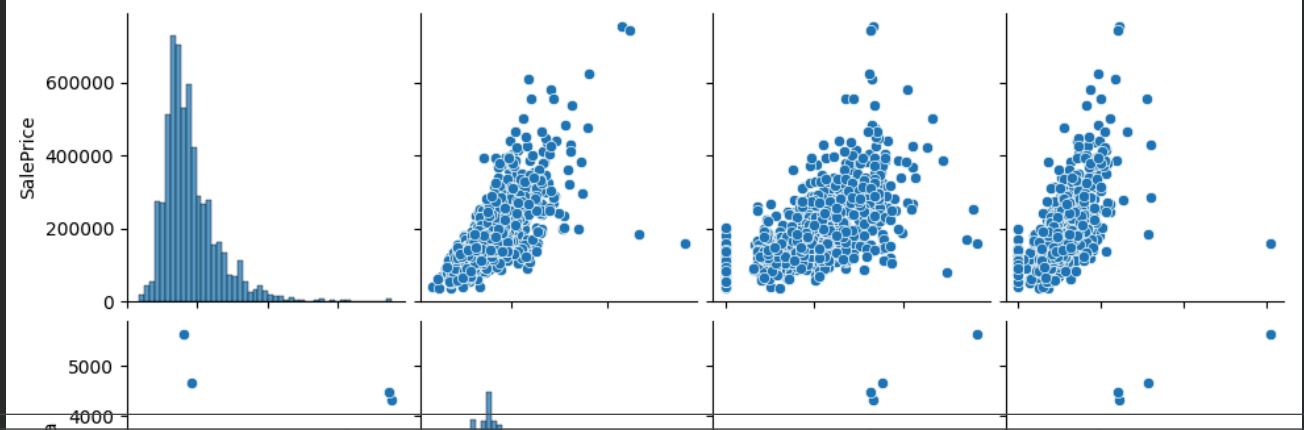
Step 10: Multivariate Analysis

```
selected_cols = [
    'SalePrice',
    -
]

sns.pairplot(
    df[selected_cols]
)

# Write all the insights of this graph
```

```
<seaborn.axisgrid.PairGrid at 0x7d929fdc4fe0>
```



```
corr = df.select_dtypes(
    include=['float', 'int']
).corr()

plt.figure(figsize=(15,10))

sns.heatmap(
    corr,
    cmap='coolwarm',
    annot=True
)

# Find which column is important for the model training..
# Find all the insights of all the columns
```

